

PRACTICAL WEEK - 7

TASK 1: Write a program that prints the addresses Of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment

```
siddu@DESKTOP-5CRM05P: ~ X Ubuntu X
siddu@DESKTOP-5CRM05P:~$ mkdir practical_memory
siddu@DESKTOP-5CRM05P:~$ cd practical_memory
siddu@DESKTOP-5CRM05P:~/practical_memory$ pwd
/home/siddu/practical_memory
siddu@DESKTOP-5CRM05P:~/practical_memory$ nano memory_layout.c
siddu@DESKTOP-5CRM05P:~/practical_memory$ gcc memory_layout.c -o memory_layout
siddu@DESKTOP-5CRM05P:~/practical_memory$ ./memory_layout
Process ID: 1684

Code address      : 0x57c73ec321c9
Global variable   : 0x57c73ec35010
Static variable   : 0x57c73ec35014
Local static variable : 0x57c73ec35018
Heap variable     : 0x57c74ee45010
Stack variable    : 0x7ffcc9e9413c

Process is running...
Press Ctrl+C to stop.
^C
siddu@DESKTOP-5CRM05P:~/practical_memory$
```

```
GNU nano 0.7.1 memory_layout.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;
static int static_var = 20;

void code_function()
{
    printf("This is a code function.\n");
}

int main()
{
    int stack_var = 30;
    static int local_static_var = 40;

    int *heap_var = (int *)malloc(sizeof(int));
    *heap_var = 50;

    printf("Process ID: %d\n", getpid());

    printf("Code address      : %p\n", (void *)code_function);
    printf("Global variable   : %p\n", (void *)&global_var);
    printf("Static variable    : %p\n", (void *)&static_var);
    printf("Local static variable : %p\n", (void *)&local_static_var);
    printf("Heap variable      : %p\n", (void *)heap_var);
    printf("Stack variable     : %p\n", (void *)&stack_var);

    printf("\nProcess is running...\n");
    printf("Press Ctrl+C to stop.\n");

    while (1)
    {
        sleep(2);
    }

    free(heap_var);

    return 0;
}
```

```

siddhu@DESKTOP-SCRM05P: ~$ cd ~/practical_memory
siddhu@DESKTOP-SCRM05P:~/practical_memory$ nano memory_analysis.c
siddhu@DESKTOP-SCRM05P:~/practical_memory$ gcc memory_analysis.c -o memory_analysis
siddhu@DESKTOP-SCRM05P:~/practical_memory$ ./memory_analysis &
[1] 1868
Process ID: 1868
Code address      : 0x5a5676aa31c9
Global variable   : 0x5a5676aa6010
Static variable   : 0x5a5676aa6014
Local static variable : 0x5a5676aa6018
Heap variable     : 0x5a56997cd010
Stack variable    : 0x7ffd569151ec

Process is running...
siddhu@DESKTOP-SCRM05P:~/practical_memory$ cat /proc/1868/maps
5a5676aa3000-5a5676aa3000 r--p 00000000 08:30 23616 /home/shreyaj/practical_memory/memory_analysis
5a5676aa3000-5a5676aa4000 r--p 00001000 08:30 23616 /home/shreyaj/practical_memory/memory_analysis
5a5676aa4000-5a5676aa5000 r--p 00002000 08:30 23616 /home/shreyaj/practical_memory/memory_analysis
5a5676aa5000-5a5676aa6000 r--p 00003000 08:30 23616 /home/shreyaj/practical_memory/memory_analysis
5a5676aa6000-5a5676aa7000 rw-p 00003000 08:30 23616 /home/shreyaj/practical_memory/memory_analysis
5a56997c0000-5a56997ce000 rw-p 00000000 00:00 0 [heap]
746e74228000-746e74228000 r--p 00000000 08:30 13641 /usr/lib/x86_64-linux-gnu/libc.so.6
746e74228000-746e743c0000 r--p 00028000 08:30 13641 /usr/lib/x86_64-linux-gnu/libc.so.6
746e743c0000-746e7441a000 r--p 001c0000 08:30 13641 /usr/lib/x86_64-linux-gnu/libc.so.6
746e7441a000-746e74412000 r--p 002b0000 08:30 13641 /usr/lib/x86_64-linux-gnu/libc.so.6
746e74412000-746e74414000 rw-p 002b1000 08:30 13641 /usr/lib/x86_64-linux-gnu/libc.so.6
746e74454000-746e74421000 rw-p 00000000 00:00 0
746e74454000-746e74457000 rw-p 00000000 00:00 0
746e74457000-746e74458000 r--p 00000000 00:00 0
746e74458000-746e74468000 r--p 00000000 00:00 0
746e74468000-746e74463000 r--p 00000000 00:00 0
746e74463000-746e74465000 r--p 00000000 00:00 0
746e74465000-746e74467000 r--p 00003000 08:30 13638 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
746e74467000-746e74497000 r--p 00003000 08:30 13638 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
746e74497000-746e744a4000 r--p 00003000 08:30 13638 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
746e744a4000-746e744a5000 rw-p 00030000 08:30 13638 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
746e744a5000-746e744a5000 rw-p 00030000 08:30 13638 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
746e744a5000-746e744a6000 rw-p 00000000 00:00 0
7ffd568f6000-7ffd56917000 rw-p 00000000 00:00 0 [stack]
siddhu@DESKTOP-SCRM05P:~/practical_memory$

```

TASK 2: Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```

GNU nano 8.7.1 memory_analysis.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;
static int static_var = 20;

void display()
{
    printf("Inside code segment\n");
}

int main()
{
    int stack_var = 30;
    static int local_static = 40;

    int *heap_var = malloc(sizeof(int));
    *heap_var = 50;

    printf("Process ID: %d\n", getpid());

    printf("Code address      : %p\n", (void *)display);
    printf("Global variable   : %p\n", (void *)&global_var);
    printf("Static variable     : %p\n", (void *)&static_var);
    printf("Local static variable : %p\n", (void *)&local_static);
    printf("Heap variable       : %p\n", (void *)heap_var);
    printf("Stack variable      : %p\n", (void *)&stack_var);

    printf("\nProcess is running...\n");

    while (1)
    {
        sleep(2);
    }

    free(heap_var);

    return 0;
}

```